ITE183 – Web Systems and Technologies

Laboratory Activity 2: WebSocket Server Report

Lady Antonia D. Tugahan
ITE183 - IT4D.1

I. Objectives

The purpose of this activity is to create a WebSocket server using Bonezegei Scripting Language (BSL), handle client connections and disconnections, and allow messages to be shared between multiple browser windows in real time.

II. Tools and Files Used

I used Bonezegei Scripting Language v1.3.3, a terminal, a web browser, and GitHub. The project contains app.bzg for the server, package.json for its settings and dependency, public/index.html for the chat page, and public/404.html for the page-not-found message. I also included a README, MIT License, and .gitattributes file for the GitHub repository.

III. Procedure

Project setup: I created a public GitHub repository named ITE183_Lab2_Websocket and added the project files. I kept the source code in app.bzg and the website files inside the public folder.

Installing and starting the server: I checked the Bonezegei version with bonezegei -v, installed the required dependency with bzg install, and started the application using bzg run.

Testing the chat: I opened the chat page in two browser windows, connected both clients to the server, and sent messages from each window to check whether the messages reached both clients.

Checking the missing-page route: I opened a URL that is not defined by the application to check whether the custom 404 page appears.

IV. Results and Screenshots

The screenshots below should be taken from my own computer after running the application. I will replace each placeholder with the actual result before submitting this report.

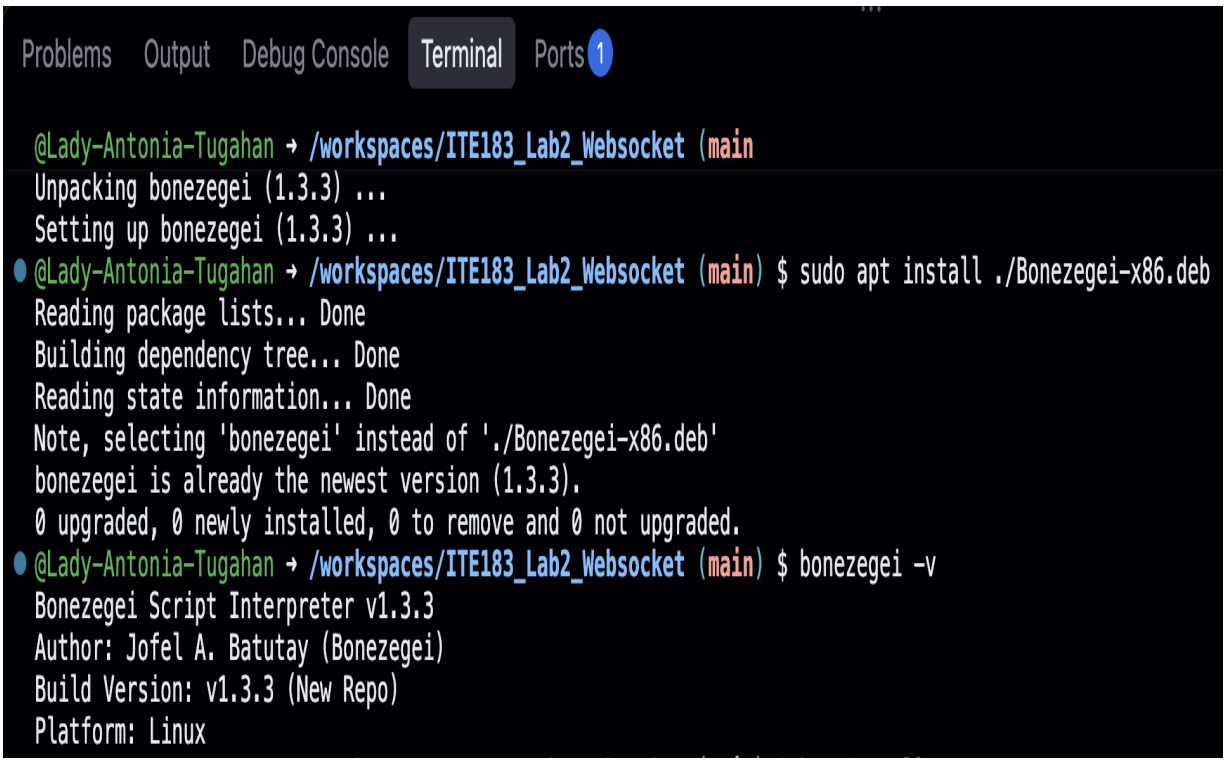


Figure 1. Bonezegei version

Observation: The version verification shows whether the required version of Bonezegei is installed. Version 1.3.3 is needed for this activity because the WebSocket server uses its networking features.

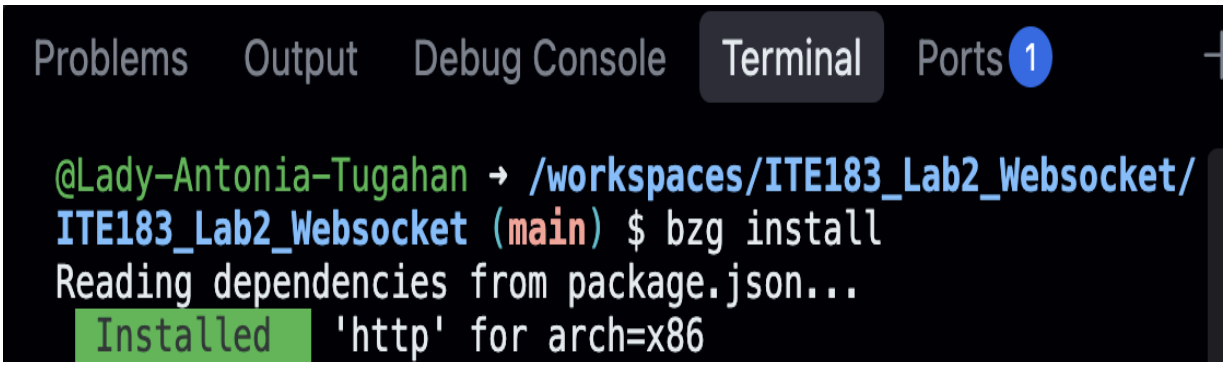


Figure 2. Dependency installation

Observation: The package installation prepares the project to use the HTTP server library. This library allows the application to handle HTTP requests and WebSocket connections.

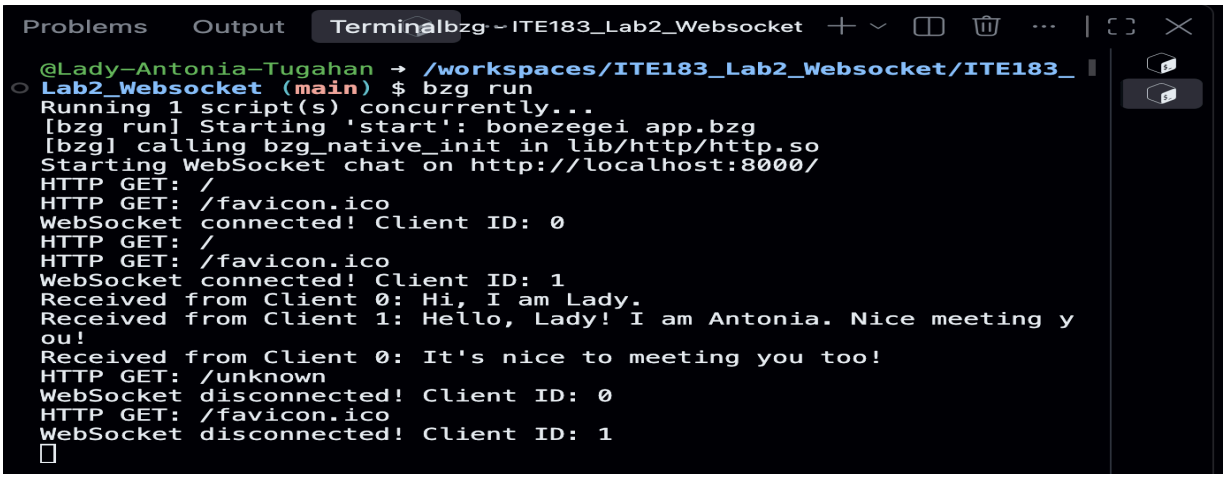


Figure 3. Server execution

Observation: The terminal logs help show what is happening on the server. Each connected client is assigned an ID, which allows the server to identify the connection. When a client sends a message, the server receives it through the onData event handler and broadcasts it to connected clients using server.sendAll(). In addition, the terminal logs incoming HTTP GET requests for /, /favicon.ico, and /unknown.

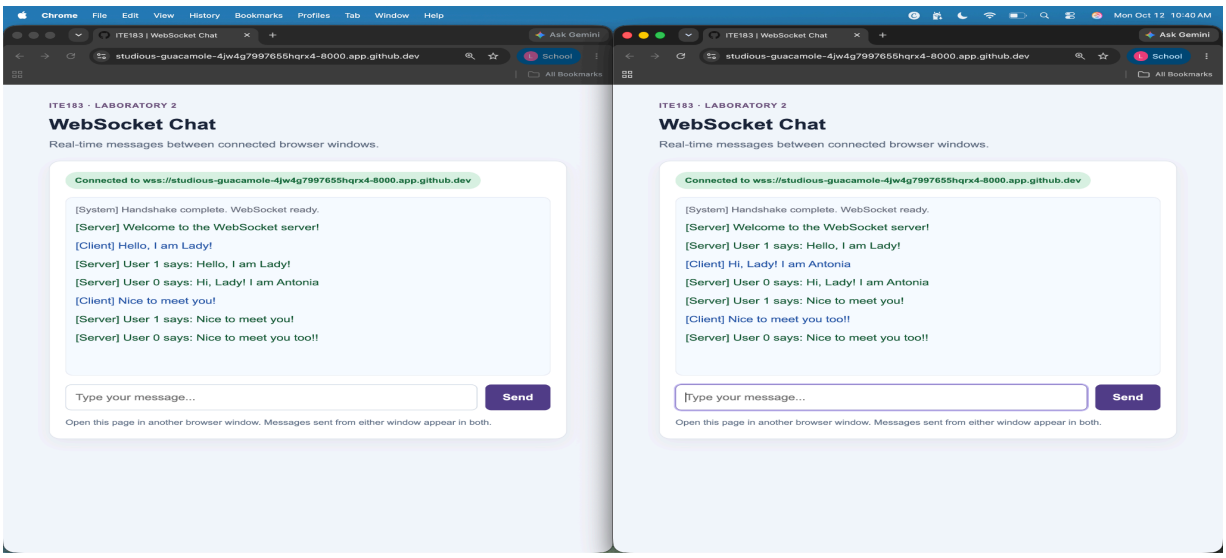


Figure 4. Multi-client chat

Observation: The chat interface demonstrates real-time communication between multiple clients. Both browser windows use the same server, allowing messages to be shared immediately. The WebSocket connection remains active while the clients are connected, so users do not need to reload the page to receive new messages.

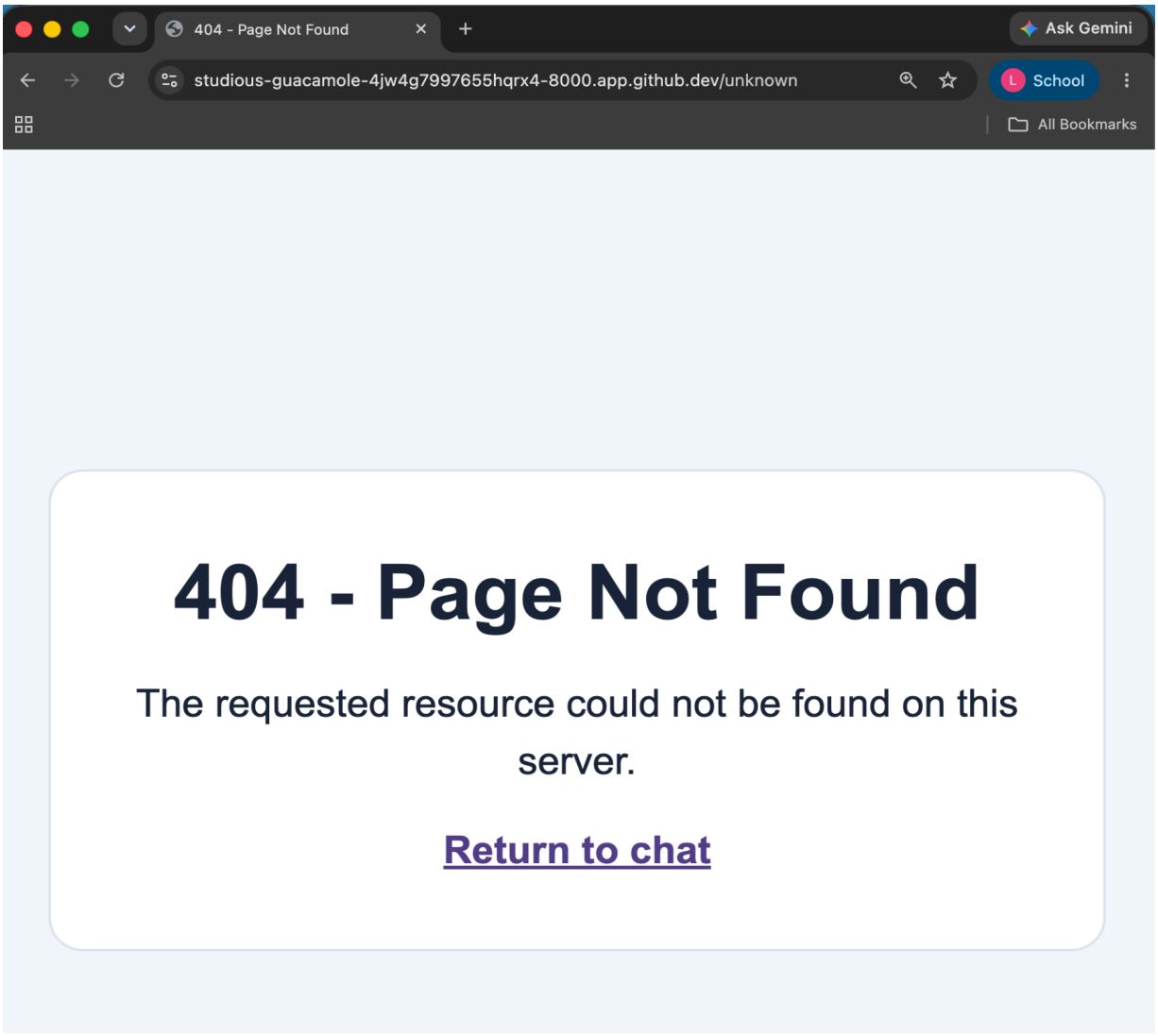


Figure 5. Custom 404 page

Observation: The server checks the requested path through the HTTP GET handler. When the path is /, it displays index.html. For other paths, it serves 404.html, which informs the user that the requested page could not be found.

V. Discussion

In this activity, I worked with the Bonezegei Scripting Language to create a WebSocket server that supports real-time communication between multiple browser clients. The application uses an HTTP server to display the chat interface and a WebSocket connection to handle messages.

One important part of the activity is understanding the difference between HTTP and WebSocket communication. HTTP is used when the browser requests the webpage, while WebSocket allows the browser and server to continue exchanging messages after the connection is established.

The server also uses different event handlers to manage client activity. The onConnect handler runs when a client connects, onData handles incoming messages, and onDisconnect records when a client leaves. These handlers allow the server to respond to client activity without requiring users to refresh their browsers.

The main feature of the application is message broadcasting. Instead of sending a response only to the client who sent the message, the server uses server.sendAll() to send it to every active connection. This is what allows multiple browser windows to participate in the same chat.

The activity also includes basic HTTP request handling. The server displays the main chat interface for the home route and a custom 404 page for undefined routes.

VI. Conclusion

This laboratory activity helped me understand how a WebSocket server works and how it can be used for real-time communication. I learned how the server handles client connections, receives messages, broadcasts them to connected users, and detects disconnections.

I also became more familiar with using Bonezegei to create an HTTP server, serve HTML files, and handle different routes. Testing the application using multiple browser windows is a useful way to see how WebSocket communication works.

Overall, the activity demonstrates how WebSocket can be used to build applications that need continuous communication between the server and connected clients, such as chat systems and other real-time applications.

VII. Repository

GitHub repository: https://github.com/Lady-Antonia-Tugahan/ITE183_Lab2_Websocket